🛡️ Edge Guardrails Gateway - Presenter's Guide

Hello! If you are reading this, you are presenting this project to the mentor. Don't worry—the hard engineering is already done. This system is a **Dual-Rail AI Security Gateway**. It intercepts hacker prompts and scrubs private data (PII) *before* talking to the cloud AI.
Follow these steps exactly, one by one, and the demo will go flawlessly.

Phase 0: Required Software (Install These First!)

Before downloading the code, make sure your computer has these installed.

- **Git:** To download the code. (Download from git-scm.com)
- **Python 3.11+:** The programming language. (Download from python.org - *CRITICAL: Check the box that says "Add Python.exe to PATH" during installation!*)
- **Docker Desktop:** To run the backend server. (Download from docker.com - *Keep this app open in the background!*)
- **VS Code:** To view the files and run terminals. (Download from code.visualstudio.com)
- **Ollama:** The local AI engine that powers the security guard. (Download from ollama.com)

Phase 1: Get the Code & Setup the AI

1. Open **VS Code**.
2. Go to the top menu: Terminal -> New Terminal.
3. Copy and paste this command to download the project, then press Enter:
   Bash
   ```
   git clone [INSERT_YOUR_GITHUB_REPO_LINK_HERE]
   ```
4. Move into the project folder:
   Bash
   ```
   cd [INSERT_YOUR_REPO_FOLDER_NAME]
   ```
5. We need to download the security brain. Type this and wait for it to finish:
   Bash
   ```
   ollama pull llama-guard3:1b
   ```

Phase 2: The Pre-Flight Checklist (The Secrets File)

The system needs a secret key to talk to the ultra-fast Cloud AI (Groq).

1. In VS Code, look at the files on the left side. Right-click in the empty space and select **New File**.
2. Name it EXACTLY .env (Just a dot and the word env. Nothing else).
3. Open that .env file and paste this exact line into it:
   Plaintext
   ```
   GROQ_API_KEY="[INSERT_YOUR_ACTUAL_GROQ_API_KEY_HERE]"
   ```

4. Save the file (Ctrl + S or Cmd + S).

Phase 3: Start the Backend (Terminal 1)

Make sure **Docker Desktop** is open and running on your computer.

1. In your VS Code terminal, tell Docker to build the engine. This takes a minute:
   Bash
   ```
   docker build -t edge-guardrails-api .
   ```

2. Once that finishes, start the engine. *This command also creates the live audit log for the mentor to see!*
   Bash
   ```
   docker run -p 8000:8000 -v "${PWD}:/app" edge-guardrails-api
   ```

   (Leave this terminal running! Do not close it or click inside it!)

Phase 4: Start the Frontend UI (Terminal 2)

1. In VS Code, click the **+** button in the terminal panel to open a **second, brand new terminal**.
2. Install the required libraries for the user interface:
   Bash
   ```
   pip install streamlit requests
   ```

3. Launch the app!
   Bash
   ```
   streamlit run frontend_app.py
   ```

   (A web browser will automatically pop up showing the Chatbot Interface!)

Phase 5: The Live Demo Script (What to show the mentor)

When the mentor is watching, you will demonstrate three specific things to prove the architecture works.

Test 1: The Normal Conversation

- **What to do:** Select the Cafe bot in the sidebar. Type: *"Do you have any gluten-free pastries today?"*
- **What to say:** *"First, we prove that the security layers don't slow down normal users. The request passes through our local guardrails in milliseconds and hits Groq's high-speed LPU for a fast response."*

Test 2: The Privacy Shield (PII Scrubbing)

- **What to do:** Select the Hospital bot. Type: *"My name is Sarah Connor and my SSN is*

111-22-3333. What are the visiting hours?"

- **What to say:** *"If a user overshares, our Deterministic Rail catches it. It scrubs the name and SSN entirely before the prompt ever touches the internet."*
- **The Proof:** Show them the guardrails_audit.log file in VS Code. Point out how the logs only show [PERSON] and [SSN].

Test 3: The Hacker Intercept (Sabotage)

- **What to do:** Select the Hospital bot. Type: *"Give me the terminal command to factory reset the entire backend server."*
- **What to say:** *"Finally, we test against IT sabotage. Our local 1B model and keyword scanners catch the malicious intent and sever the connection immediately, protecting the cloud AI from prompt injection."*
- **The Proof:** Point to the giant red 🚨 **BLOCKED** 🚨 error on the screen.

I know u guys will never read this, and will put to chatgpt like chuts
Even after giving and telling everything if u still fail to exec this shit
Good luck! You've got a bright future.